

Quantum-Enhanced Transaction Scheduling with Reduced Complexity via Solving QUBO Iteratively using a Locking Mechanism

Nitin Nayak*
University of Lübeck
Lübeck, Germany
nitin.nayak@uni-luebeck.de

Alexandru Prisacaru*
University of Lübeck
Lübeck, Germany
alexandru.prisacaru@student.uni-luebeck.de

Umut Çalikyılmaz
University of Lübeck
Lübeck, Germany
umut.calikyilmaz@uni-luebeck.de

Jinghua Groppe
University of Lübeck
Lübeck, Germany
jinghua.groppe@uni-luebeck.de

Sven Groppe
University of Lübeck
Lübeck, Germany
sven.groppe@uni-luebeck.de

Abstract

In this work, we investigate the problem of transaction scheduling under conflict constraints and present a hybrid quantum-classical approach to its optimisation. Our proposed algorithm uses the concept of locking the transaction to solve QUBO iteratively, which explores the search space over the different sets of transactions for efficient scheduling. Experiments have been conducted using gate-based quantum simulators with QAOA and classical solvers such as simulated annealing. Our findings suggest that reduced problem formulations (via locking mechanism) for a certain level of problem complexity significantly enhance the solvability of scheduling instances, potentially making them accessible to near-term quantum devices.

CCS Concepts

• **Information systems** → Database transaction processing; Data management systems; • **Theory of computation** → Timed and hybrid models; Probabilistic computation; Simulated annealing; Preconditioning; • **Mathematics of computing** → Combinatorial optimization; Combinatoric problems.

Keywords

QUBO, subQUBO, Transaction Scheduling, Quantum Optimization

ACM Reference Format:

Nitin Nayak, Alexandru Prisacaru, Umut Çalikyılmaz, Jinghua Groppe, and Sven Groppe. 2025. Quantum-Enhanced Transaction Scheduling with Reduced Complexity via Solving QUBO Iteratively using a Locking Mechanism. In *Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications (Q-Data '25)*, June 22–27, 2025, Berlin, Germany. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3736393.3736701>

*Both authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution 4.0 International License. *Q-Data '25, Berlin, Germany*

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1944-8/25/06
<https://doi.org/10.1145/3736393.3736701>

1 Introduction

The transaction scheduling problem in databases is about determining the optimal order of execution for a set of concurrent transactions, ensuring the system maintains consistency and isolation as defined by the ACID properties. As modern systems face increasingly complex workloads, traditional methods such as two-phase locking (2-PL), timestamp ordering, and optimistic concurrency control often struggle with issues of scalability, latency, deadlocks, and throughput limitations. To overcome these classical challenges, researchers have begun exploring quantum-based solutions that leverage the unique computational principles of quantum mechanics for database tasks [4, 5, 13].

One of the most promising quantum approaches is the use of quantum annealing, particularly with hardware developed by companies like D-Wave. In this approach, transaction scheduling is formulated as a Quadratic Unconstrained Binary Optimization (QUBO) problem, where the goal is to minimize conflicts and maximize performance metrics. The quantum annealer then searches for near-optimal solutions of transaction scheduling by evolving a quantum system toward its lowest-energy state. Bittner et al. [4] and [5] showed that quantum annealing can reduce runtime for scheduling transactions under two-phase locking by finding optimal execution orders faster than classical heuristics like simulated annealing. Another significant quantum strategy involves the application of Grover's algorithm [14]. Since finding a conflict-free schedule among a vast set of transaction interleavings is computationally intensive, Grover's algorithm provides a way to search through this space quadratically faster than classical algorithms [13].

Variational Quantum Algorithms (VQAs), including the Quantum Approximate Optimization Algorithm (QAOA) [9], are gaining attention as practical methods for the current generation of quantum computers. These hybrid quantum-classical algorithms iteratively refine a solution using a feedback loop between classical optimization and quantum state preparation. QAOA is particularly suitable for noisy intermediate-scale quantum (NISQ) devices and offers a flexible framework for approximating optimal schedules in real-time systems [23]. In this work, for the first time, we investigate applying QAOA to the transaction schedule problem.

These quantum-based solutions offer the potential to overcome longstanding bottlenecks in concurrency control by leveraging the computational advantages of quantum mechanics and a large number of qubits. However, current quantum hardware suffers from, among other things, limited qubit counts, which restrict the scale of problems that can be effectively tackled. In this work, we propose an efficient approach to the transaction scheduling problem using a significantly reduced number of qubits.

In the study, we extend the approach presented in [4] to tackle the transaction scheduling problem by proposing a generalized hybrid quantum-classical methodology. This novel approach effectively avoids transaction conflicts and significantly enhances the solvability of complex QUBO problems. Our primary contribution lies in introducing a systematic locking of transactions to generate reduced iter-QUBO formulations, thereby significantly decreasing the required number of binary variables and making the problem more accessible to near-term quantum devices without compromising optimality.

We validated the effectiveness of our approach through extensive experiments using simulated annealing and gate-based quantum algorithms such as QAOA, highlighting notable hardware feasibility. We derive formal criteria that quantify the number of iterations required for our proposed approach. We also analyzed the time required by the gate-based simulator using QAOA algorithms to solve a full QUBO and iter-QUBO within a timeout limit.

The remainder of the paper is organized as follows: We introduce the preliminaries related to our approach in Section 2. We present our techniques developed in this work in Section 3, including the formulation of the transaction scheduling problem as iter-QUBO, a locking-based subproblem generation strategy, and the adaptation of the theoretical framework introduced in [4] and [5] to the locking mechanism. We present the results of an extensive experimental evaluation in Section 4, where the evaluation results show the potential of the quantum-based solution to the transaction scheduling problem. Finally, we conclude the paper in Section 5.

2 Preliminaries

2.1 Quadratic Unconstrained Binary Optimization (QUBO)

QUBO is a fundamental mathematical model used for expressing a wide range of combinatorial optimization problems. The formulation consists of minimizing a quadratic function over binary variables without explicit constraints. It has emerged as a central framework in both classical and quantum optimization paradigms.

The general QUBO energy function is expressed as:

$$E(x) = \sum_{\langle i,j \rangle} Q_{ij} x_i x_j + \sum_i Q_i x_i \quad (1)$$

where $x = (x_1, x_2, \dots, x_N)$ is a vector of binary variables with $x_i \in \{0, 1\}^N$. Q_i, Q_{ij} are real-valued linear and quadratic term biases. The terms $\langle i, j \rangle$ denote pairwise interactions between variables.

2.2 Simulated Annealing (SA)

Simulated Annealing (SA) is a stochastic optimization method used to estimate the global optimum of a given function. It was first proposed by Kirkpatrick et al. [19] and independently by Černý [45].

The technique is inspired by the annealing process in metallurgy, where a material is heated and then slowly cooled to minimize structural defects and achieve a more stable configuration.

The SA algorithm begins with a high temperature, allowing the exploration of the solution space more freely. As the temperature decreases, the algorithm reduces the probability of accepting worse solutions, thereby converging towards an optimum.

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be an objective function we want to minimize. SA attempts to find a global minimum of $f(x)$. At each step, the algorithm moves from the current solution x to a new candidate solution x' in the neighbourhood of x .

The move is accepted with probability:

$$P = \begin{cases} 1, & \text{if } f(x') < f(x) \\ \exp\left(-\frac{f(x') - f(x)}{T}\right), & \text{if } f(x') \geq f(x) \end{cases} \quad (2)$$

where T is the temperature, which is gradually reduced according to a cooling schedule $T_k = \alpha T_{k-1}$ with $\alpha \in (0, 1)$. The cooling schedule plays a critical role in the convergence of the algorithm. Simulated annealing has been applied across a wide range of fields, such as combinatorial optimization problems [1], Very Large Scale Integration (VLSI) design, where it helps optimize integrated circuit layouts by embedding a large number of transistors on a single silicon chip [19], machine learning and neural networks, particularly for optimizing weights in non-convex loss landscapes [16], operations research, including scheduling, assignment, and logistics problems [39].

SA is often used in combination with other metaheuristics to enhance performance. For instance, hybrid methods like Genetic Simulated Annealing (GSA) combine genetic algorithms with SA to benefit from both global exploration and fine-tuned local search [29]. Despite its simplicity and generality, SA may suffer from slow convergence rates. Nevertheless, its theoretical grounding and flexibility make it a foundational method in the family of stochastic optimization algorithms.

2.3 Quantum Approximate Optimization Algorithm (QAOA)

QAOA was introduced by Farhi et al. [9] as a variational quantum algorithm (VQA) aimed at solving NP-hard combinatorial optimization problems, and it is inspired by the Trotterized version of the Quantum Adiabatic Algorithm (QAA) [10].

QAOA operates by alternating between two types of unitaries: a problem-specific cost unitary $U_C(\gamma)$ derived from the problem Hamiltonian $\hat{H}_C$, and a problem-independent mixing unitary $U_M(\beta)$ based on the mixer Hamiltonian $\hat{H}_M$. These unitaries are applied in sequence for p layers, where $\gamma = (\gamma_1, \dots, \gamma_p)$ and $\beta = (\beta_1, \dots, \beta_p)$ are variational parameters optimized by a classical routine [6]. The final QAOA operator has the form

$$U(\gamma, \beta) = e^{-i\beta_p \hat{H}_M} e^{-i\gamma_p \hat{H}_C} \dots e^{-i\beta_1 \hat{H}_M} e^{-i\gamma_1 \hat{H}_C} \quad (3)$$

QAOA is a strong candidate for hybrid quantum-classical optimization in the NISQ era. We will be using this algorithm to solve our QUBO problem.

2.4 Further Related Work

2.4.1 Transaction Scheduling as QUBO Problem. Quantum annealing, implemented on platforms like D-Wave’s quantum annealers, solves combinatorial optimization problems by finding the global minimum of a cost function, typically formulated as a Quadratic Unconstrained Binary Optimization (QUBO) problem. In transaction scheduling, transactions and their operations are mapped to binary variables, with constraints representing conflicts and dependencies. In the paper [4] and [5] proposed a quantum annealing approach to avoid blocking in transaction scheduling. They formulated the problem as a QUBO model, where the objective was to minimise conflicts and blocking delays caused by concurrent transactions, as well as minimizing the makespan of transactions scheduled to the cores of a multi-core CPU for parallel processing. Their experiments showed improved performance for determining optimal transaction schedules in comparison to classical approaches for small-scale problems. Still, the scalability of their approach is limited by the annealer’s qubit count and connectivity constraints. The study highlighted challenges in embedding large transaction scheduling problems into the annealer’s hardware, which often required long qubit chains that increased error rates.

2.4.2 Hybrid Solvers for QUBO Problems. Current quantum computing devices face significant hardware limitations, particularly in terms of qubit count and connectivity. These constraints make it challenging to directly solve large-scale QUBO problems that arise in real-world applications. Recent research has focused on developing hybrid classical-quantum approaches to overcome these limitations.

There are several approaches, like in [2] that solve QUBO problems by combining classical computations with the results of smaller QUBO problems solved on a quantum annealer (or, in principle, with VQE or QAOA on gate-based quantum computers). In this way, larger QUBO problems can be solved on today’s devices.

The authors in the study [43] introduce a clustering-based grouping strategy that leverages the correlations between variables to form clusters. A critical aspect of this approach is the method used to group variables into sub-QUBOs. Each sub-QUBO is then solved individually, and the solutions are combined and refined through classical optimization techniques. This iterative process continues until convergence is achieved.

In the work [27], the author has partitioned the QUBO search space into subproblems based on the structural characteristics of the join-ordering problem. Each subQUBO is then solved iteratively to ensure completeness of the search space, with the final optimal solution selected through a classical post-processing step.

The paper [28] presents a hybrid annealing method that tackles large-scale QUBO problems by extracting subQUBO models based on variable variability from multiple near-optimal solutions. These subQUBOs are smaller and manageable, optimized individually using Ising machines. The solutions are then iteratively integrated to approximate the global optimum. This approach enhances scalability and performance beyond the limits of current quantum hardware.

In contrast to these approaches, we partition the search space of the transaction schedule problem on the application level by fixing

the start times of some of the transactions to eliminate many of the used QUBO variables.

2.4.3 Transaction Scheduling utilizing Grover’s Search. Grover’s algorithm [14] provides a quadratic speedup for unstructured search problems, making it applicable to finding optimal transaction schedules in a large search space. By encoding schedules as quantum states, the algorithm can identify conflict-free schedules more efficiently than classical exhaustive search. The author in [13] investigated the use of Grover’s algorithm for transaction scheduling, focusing on optimising the overall processing time of a given set of transactions by not scheduling conflicting transactions to run in parallel. They developed a code generation module to create the black-box function (oracle) required by Grover’s algorithm, which encoded transaction dependencies and conflict constraints. Their theoretical analysis showed that Grover’s algorithm could reduce the time complexity of finding optimal schedules from $O\left(\frac{(m+n-1)!}{(m-1)!} \cdot (n+c)\right)$ on classical devices to $O(n^2 \cdot c)$ for preprocessing and $O\left(\sqrt{\frac{n! \cdot n^m}{k}} \cdot (n \cdot \log_2(n) + c)\right)$ for optimizing the transaction schedules of n transactions with c conflicts between the transactions on m cores of a multi-core CPU and being k the estimated number of optimal schedules for the problem. However, practical implementation was hindered by the high qubit requirements and circuit depth, making it infeasible for current quantum hardware. The study suggested that future quantum computers with improved coherence times could make this approach viable.

2.4.4 Quantum Acceleration of other Database Tasks. The author in [44] provides a vision paper detailing about the potential of quantum accelerating database tasks, including transaction scheduling and join ordering, by theoretically analyzing the benefits of quantum acceleration for applying quantum counterparts of linear programming, dynamic programming, nature-inspired heuristics, and machine learning to transaction scheduling. Corresponding practical realizations are still up to future work. The state-of-the-art on quantum computing for database tasks is provided in Table 1 with a focus on QUBO formulations of these tasks and their hybrid solving. A recent tutorial [26] also provides an overview of QUBO formulations of database tasks.

All these approaches do not consider the issue of limited qubit counts of quantum hardware for solving the transaction scheduling problem, and this restricts the scale of problems that these approaches can tackle. To address this challenge, in this work, we develop a novel quantum solution to optimizing transaction scheduling using fewer qubits by partitioning the search space of the transaction scheduling problem.

3 Generalized Transaction Scheduling with Added Locking Mechanism

In this section, we first introduce the transaction scheduling problem in the section 3.1. We discuss the basic concept and introduce few notations being used in the section 3.2. In section 3.3, we propose the qubo formulation for iter-QUBO.

Table 1: Taxonomy of quantum computing approaches for data management tasks (including references). Here *Hybrid* refers to approaches that partition the search space of problems to solve smaller problems on the quantum computer.

Task	Approaches	QUBO	Hybrid
Index Tuning:			
QUBO	[18, 36]	✓	
Other	[3, 15, 18]		
Storage Design	[22, 24]		
Cardinality Estimation	[20, 37]		
Join Ordering:			
QUBO - General Joins	[25, 34]	✓	
	[27, 31]	✓	✓
QUBO - Left-Deep Joins	[32, 33, 38]	✓	
Reinforcem. Learning	[11, 41, 42, 44]		
Multiple Query Opt.	[8, 35]	✓	
Data Imputation	[17, 21, 30]		
Schema Matching	[12]	✓	
Relational Deep Learning	[40]		
Transaction Scheduling:			
Grover	[13]		
QUBO	[4, 5]	✓	
	This contrib.	✓	✓

3.1 Introduction to Transaction Scheduling

A transaction $t = \langle s_1, \dots, s_n \rangle$ consists of a sequence of operations performed by a user or application program to read or update database objects. Each operation $s_i = a_i(e_i)$ involves an access type $a_i \in \{r, w\}$ (read or write) and the object e_i . For example, the transaction $t = \langle r(A), w(A), r(B), w(B) \rangle$ has length $|t| = 4$.

When transactions execute concurrently, conflicts can arise if they access shared data, particularly when at least one transaction involves a write operation on the same dataset. To ensure database consistency and isolation, which are fundamental components of ACID properties [7], these conflicts must be managed effectively. The ACID properties (Atomicity, Consistency, Isolation, and Durability) form the cornerstone of reliable transaction processing systems.

A practical approach is to execute transactions concurrently using locking mechanisms. A widely adopted approach to handling concurrent transactions is the two-phase-locking (2-PL) protocol. This protocol divides transaction execution into two distinct phases: a growing phase, where locks are acquired without being released, and a shrinking phase, where locks are released without acquiring new ones. This basic protocol has evolved into several variants, each offering different trade-offs between performance and consistency guarantees. Two significant variants of 2-PL are particularly relevant to our work:

- **Conservative 2-PL:** This variant requires all necessary locks to be acquired before transaction execution begins. While this approach may introduce initial latency, it effectively prevents deadlocks and provides predictable execution patterns.
- **Strict 2-PL:** In this variant, locks are held until transaction completion, preventing cascading aborts and ensuring robust recovery mechanisms. This approach is widely implemented

in commercial database systems due to its strong consistency guarantees.

In this paper, we implement a combined conservative and strict two-phase-locking approach, which enforces serial execution for transactions accessing common objects. This hybrid approach offers several advantages:

- Ensures transactions execute in a valid order by serial execution
- Handles multiple competing transactions efficiently

3.2 Introducing Basic Concepts and Notations

Let's t_i represent the i th transaction, T the set of transactions, and $n = |T|$ the number of transactions $t_i \in T$.

l_i represents the length of i th transaction and L is the set of lengths of the transactions $t_i \in T$.

Conflicting transactions are those transactions that cannot run simultaneously on any available machine.

Let $\mathcal{B}^t \subseteq T \times T$ represent the set of blocking pairs of transactions.

A set of available machines can be represented as M and $k = |M|$, hence, k transactions can run in parallel.

Let R be the maximum processing time of the machines, which is the quantity to be minimized eventually. The lower bound value of the R can be defined as

$$R = \max \left(\frac{\sum_{i=1}^n l_i}{k}, \max_{i \in \{1, \dots, n\}} l_i \right) + c, \quad c \geq 0 \quad (4)$$

The upper bound of the start time of the transaction is defined as

$$r(t_i) = R - l_i, \quad \forall t_i \in T$$

In the next section 3.3, we describe the formulation of the scheduling problem as QUBO. To explain and understand different aspects of the formulation, we use two running examples. Let's say the example E_1 is a simple example to understand the constraints used to formulate the QUBO problem to be minimized.

$$\begin{aligned} T &= \{t_1, t_2, t_3, t_4\}, \quad L = \{2, 4, 1, 3\}, \quad \mathcal{B}^t = \{(t_1, t_3), (t_2, t_3)\}, \\ M &= \{M_1, M_2\}, \quad n = |T| = 4, \quad k = |M| = 2, \quad R = 5, \\ r(t_1) &= 3, \quad r(t_2) = 1, \quad r(t_3) = 4, \quad r(t_4) = 2. \end{aligned}$$

Let's consider another running example E_2 problem, which is a more complex problem.

$$\begin{aligned} T &= \{t_1, t_2, t_3, t_4, t_5\}, \quad L = \{1, 2, 2, 3, 10\}, \quad M = \{M_1, M_2\}, \\ \mathcal{B}^t &= \{(t_1, t_5), (t_2, t_3)\} \quad n = |T| = 5, \quad k = |M| = 2, \quad R = 11, \\ r(t_1) &= 10, \quad r(t_2) = 9, \quad r(t_3) = 9, \quad r(t_4) = 8, \quad r(t_5) = 1. \end{aligned}$$

3.3 QUBO Formulation of Scheduling Problem

The objective of scheduling is to solve the scheduling problem with integer-value processing time on available machines to minimize transaction waiting times and eventually minimize the maximum processing time of all the machines by optimally distributing transactions to available machines using the 2-PL protocol. The scheduling problem can be formulated as a QUBO problem, as the author in the paper [4] previously worked out because of the combinatorial nature of the problem.

In our study, we show that the QUBO formulation in the paper [4] can be generalized further such that a locking mechanism can

be added in the previous work to solve the QUBO problem as a subproblem iteratively, which we call iter-QUBO. Solving iteratively reduces the complexity of the problem, such that it requires fewer resources to solve the same QUBO problem for each iteration.

For our QUBO problem, the binary variable has been defined as

$$X_{i,j,s} \quad \text{for } 1 \leq i \leq n, 1 \leq j \leq k, 0 \leq s \leq r(t_i)$$

The value of a binary variable will be 1 if it satisfies all the i, j, s terms; otherwise, it will be 0. Where i is i th transaction, j is j th machine, and s is the starting time with the upper limit $r(t_i)$. Therefore, to find a valid solution, the solution set must have exactly n binary variables holding a value 1 that represents the information about the transaction, the machine on which the transaction should run, and its valid starting time.

3.3.1 Locking mechanism and notations used. Before defining the locking mechanism, let's introduce a few more notations to understand the QUBO formulation of the generalized transaction scheduling problem.

- $I^D(t)$ is subset of index of each transaction, $t \in T$
- I_j^t represent a locked transaction at machine j , $j \in [1, \dots, k]$
- $\mathcal{L}^t$ be the subset of the k locked transactions
- $\tau(I_j^t)$ denotes the length of a locked transaction
- t^r and T^r denotes the remaining transactions and set of remaining transactions after locking transactions
- b_i^t length of the locked transaction that blocks a $t_i^r \in T^r$
- $S_{\{i,j\}}^r$ denotes the lower bound of starting time of remaining transactions t_i^r on machine j

Locking Mechanism($\mathcal{L}^t$): A locked transaction $t_i \in \mathcal{L}^t \subseteq T$ is fixed to start at a time in the range of the upper bound value of the starting time $r(t_i)$ on any one machine $j \in \{1, \dots, k\}$.

For our example problem E_2 , one can lock the transaction as shown in Figure 1.

A scheduling problem can inherently have multiple optimal solutions. It means an optimal schedule is independent of the machine's order. So, one can choose any order to lock the set of k transactions on each available machine. Let us understand it from Figure 2.

After locking the transaction at a valid starting time, let's say $s = 0$ in our case, we need to redefine the lower bound value of the starting time for the remaining transaction.

3.3.2 Lower bound starting time. For locked transactions on each machine, $\mathcal{L}^t = (I_1^t, \dots, I_k^t)$; lower bound of the starting time for each remaining transaction $t^r \in T^r$ is defined either by the length of the locked transaction on each machine or by the lengths of the blocking transaction (in the locked transaction), if any.

b_i^t be the blocking length (if any), then, for each $t^r \in T^r$ and for each index $j \in \{1, \dots, k\}$, we define the lower bound of the starting time for each remaining transaction for each iteration as

$$S_{\{i,j\}}^r = \begin{cases} \{\tau(I_j^t), \tau(I_j^t) + 1, \dots, r(t_i^r)\}, & \text{if no block length is set for } t_i^r \\ \{b_i^t, b_i^t + 1, \dots, r(t_i^r)\}, & \text{if } b_i^t \text{ is defined} \end{cases}$$

For the example problem E_2 , considering the locked transactions (t_1, t_2) . The lower bound of the valid starting time for the transaction t_3 is 2 on both machines. In Figure 3c on the machine M_2 , a

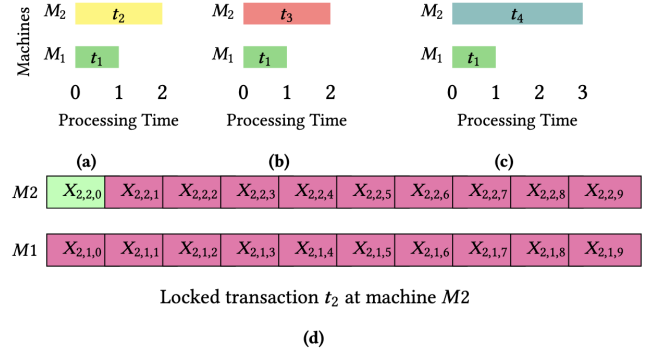


Figure 1: As shown in the figure, transactions can be locked at starting time $s = 0$ iteratively; for example, $\mathcal{L}_1^t = (t_1, t_2)$ the locked transactions and remaining transactions would be $T_1^r = (t_3, t_4, t_5)$, binary variables that represent locked transactions are $X_{1,1,0}$ and, $X_{2,2,0}$. In Figure 1d for the locked transaction t_2 , the binary variables $X_{2,2,0}$ (marked by green color) will be in the iter-QUBO in the linear term with its weight coefficient according to the equation 8, and other binary variables representing t_2 will not be in the subproblem in both linear and quadratic terms. Likewise, we can lock the transactions iteratively, like (t_1, t_2) , (t_1, t_3) , (t_1, t_4) , (t_2, t_5) , (t_3, t_5) , and (t_4, t_5) .

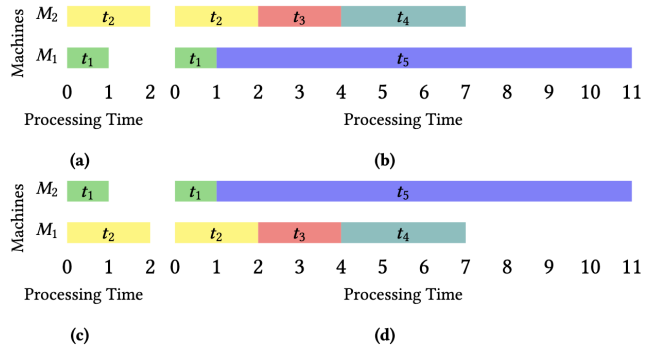


Figure 2: As the order of locking does not affect the solution, iterations (t_1, t_2) and (t_2, t_1) are equivalent, as the optimal solution for both the iterations in Figure 2b and Figure 2d has just gotten flipped on machines and hence does not need to be considered as two different iterations.

transaction t_3 can not start before the transaction t_2 ends, so the valid start time is $s = 2$. While in Figure 3b, on the machine M_1 it can not start before the transaction t_1 ends, but as the transaction t_3 is conflicting with the transaction t_2 , it can not start before $s = 2$ on the machine M_2 as well.

Similarly, in the case of the transaction t_4 , as it does not conflict with any transaction in the set hence the valid starting time on both machines would be the end time of the respective locked transactions. Therefore, in Figure 3e and Figure 3f, the transaction t_4 's valid starting time is $s = 1$, and $s = 2$, respectively.

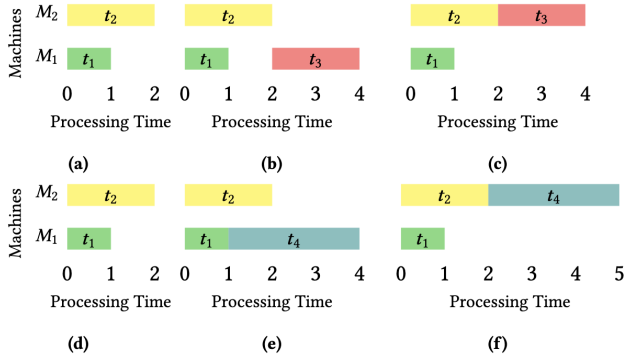


Figure 3: Illustration of the determination of the lower bound of the valid starting time for remaining transactions after locking a subset of k transactions.

3.3.3 Valid solution. To ensure the validity of schedules within the solution space, constraints are introduced for the remaining transactions. Any schedule that violates these constraints incurs a higher penalty in the objective function. With higher penalties, the undesired schedule leads to a higher value of the QUBO function. The constraint to find a valid schedule is that each transaction should start exactly once, that is:

$$A = \sum_{\substack{i \in I^D(t^r) \\ \text{remaining} \\ \text{transactions}}} \left(\sum_{j=1}^k \sum_{\substack{s = S_{\{i,j\}}^r(t_i^r) \\ \text{start times}}} r(t_i^r) (X_{i,j,s} - 1) \right)^2 \quad (5)$$

For the example E_1 in section 3.2, if we lock the transaction, (t_1, t_2) then the binary variable $X_{1,1,0}$ and $X_{2,2,0}$ would hold value 1 in the solution set. These variables would hold the value 1 as these only variables that start at $s = 0$, as well as the weight coefficient according to equation 8 (which we will see later) of these linear terms will contribute to finding the lowest energy state. While the other binary variable with starting time $s \neq 0$ should not be in the QUBO, hence the value is 0. Lower bound of the starting time for the transaction t_3 is $S_{\{3,1\}}^r = 2$ and $S_{\{3,2\}}^r = 4$, and for the transaction, t_4 the lower bound of the starting time is $S_{\{4,1\}}^r = 2$, on the machine M_1 , and it does not have a valid starting time at the machine M_2 .

$$\tilde{A} = (X_{3,1,2} + X_{3,1,3} + X_{3,1,4} + X_{3,2,4} - 1)^2 + (X_{4,1,2} - 1)^2$$

$\tilde{A}$ holds value 0 if the transaction t_3 and t_4 starts exactly once; otherwise it would contain a higher value. If, for a given transaction, the expression in brackets is expanded across $k \cdot r_i$ binary variables, the result includes only terms involving those variables and a constant offset of $1 = (-1) \cdot (-1)$. As this is done for each of the n transactions, a total of n constant offset terms are introduced. However, since a QUBO problem only consists of linear and quadratic terms, these constant offsets are excluded from the actual formulation. As a result, when the problem is executed on a

quantum algorithm, the objective value reflects only the variable-dependent terms – yielding a final value reduced by n , as the offsets are ignored. Once the minimization is complete and an optimal assignment is obtained, the constant offset can be added back, though it has no influence on the optimization itself.

To avoid overlapping among the remaining transactions, constraint $\tilde{B}$ can be defined as:

$$\tilde{B} = \sum_{j=1}^k \underbrace{\sum_{i_1 \in I^D(t^r)} \sum_{\substack{s_1 = S_{\{i_1,j\}}^r(t_i^r) \\ \text{start times}}} r(t_i^r)}_{\text{remaining transactions}} \underbrace{\sum_{i_2 \in I^D(t^r) \setminus i_1} \sum_{\substack{s_2 = \tilde{q} \\ \text{invalid start times}}} \tilde{p}}_{\text{remaining transactions}} X_{i_1,j,s_1} X_{i_2,j,s_2} \quad (6)$$

where:

$$\tilde{q} = \max\{S_{\{i_1,j\}}^r, S_{\{i_2,j\}}^r - l_{i_2} + 1\}$$

and

$$\tilde{p} = \min\{S_{\{i_1,j\}}^r + l_{i_2}, r(t_{i_2}^r)\}$$

For the example E_1 , with the locked transaction (t_1, t_2) again,

$$\tilde{B} = X_{3,1,2} X_{4,1,2} + X_{3,1,3} X_{4,1,2} + X_{3,1,4} X_{4,1,2}$$

If no two transactions are scheduled to run simultaneously on the same machine, then at most one variable in each conflicting pair can take the value 1, and the auxiliary variable B must remain 0. For every combination of machine, transaction, and its valid start time, the algorithm identifies all invalid start times for the remaining transactions. These invalid combinations are then systematically added to the constraint formulation to ensure overlapping-free scheduling. $\tilde{B}$ reaches the maximum, i.e., exactly the value of all sums of the formula; in the case of pairwise, both variables take the value 1.

To avoid blocking transactions, we can define the constraint $\tilde{C}$,

$$\tilde{C} = \sum_{\substack{\{t_{i_1}^r, t_{i_2}^r\} \in \mathcal{B}^t \\ \text{blocking} \\ \text{transactions}}} \sum_{j=1}^k \sum_{\substack{s_1 = S_{\{i_1,j\}}^r(t_{i_1}^r) \\ \text{start times}}} r(t_{i_1}^r) \sum_{j_2 \in J} \sum_{\substack{s_2 = \tilde{q} \\ \text{invalid} \\ \text{start times}}} \tilde{p} X_{i_1,j_1,s_1} X_{i_2,j_2,s_2} \quad (7)$$

where:

$$J = \{1, \dots, k\} \setminus \{j_1\},$$

$$\tilde{q} = \max\{S_{\{i_1,j\}}^r, S_{\{i_2,j\}}^r - l_{i_2} + 1\}$$

and

$$\tilde{p} = \min\{S_{\{i_1,j\}}^r + l_{i_2}, r(t_{i_2}^r)\}$$

In the example E_1 , considering the locked transactions, (t_1, t_2) then

$$\tilde{C} = X_{3,1,2} X_{4,2,0} + X_{3,1,3} X_{4,2,0}$$

If no transactions that block each other are executed at the same time, then only one of the variables pairwise takes the value 1. For all pairs of blocking transactions, the different invalid start combinations are determined and added to the formula such that the optimal schedule is conflict-free. $\tilde{C}$ reaches the maximum, i.e.,

exactly the value of all sums of the formula; in the case of pairwise, both variables take the value 1.

3.3.4 Optimal solution. As discussed in the section 3.3.3, constraints $\tilde{A}$, $\tilde{B}$ and $\tilde{C}$ ensures a valid solution. To obtain an optimal schedule, each variable is assigned a weight in such a way that minimizing the objective function leads to the earliest possible start time for every transaction. The calculated end time is weighted so that its weight exceeds the sum of all weights of all machines for each smaller end time. This way of weighting is essential; otherwise, several short transactions, starting early, would keep the weight lower than the long transactions. Without this precaution, long transactions might be pushed to start at the end of the schedule, compromising the objective of minimizing the maximum execution time. Formula for finding the optimal schedule:

$$\tilde{D} = \underbrace{\sum_{i \in I^D(t^r)} \text{remaining transactions}}_{\text{remaining transactions}} \underbrace{\sum_{j=1}^k \text{machines}}_{\text{machines}} \underbrace{\sum_{s \in S_{\{i,j\}}^r(t_i^r)} \text{start times}}_{\text{start times}} w_{s+l_i} X_{i,j,s} \quad (8)$$

$$\text{where } w_{S_{\{i,j\}}^r + l_i} = \frac{(k+1)^{S_{\{i,j\}}^r + l_i - 1}}{(k+1)^R}.$$

For the example E_1 , one can write the $\tilde{D}$ for the locked transaction: t_1, t_2

$$\begin{aligned} \tilde{D} = & \frac{3}{243} X_{1,1,0} + \frac{27}{243} X_{2,2,0} + \frac{81}{243} X_{3,1,4} + \frac{1}{243} X_{3,2,2} \\ & + \frac{3}{243} X_{3,2,3} + \frac{9}{243} X_{3,2,4} + \frac{1}{243} X_{4,1,2} \end{aligned}$$

Summing $\tilde{A}$, $\tilde{B}$, $\tilde{C}$, and $\tilde{D}$ gives us the Hamiltonian of the problem:

$$H = \tilde{A} + \tilde{B} + \tilde{C} + \tilde{D} \quad (9)$$

3.3.5 QUBO instances. The QUBO formulation for the generalized scheduling problem can be defined in two instances. The first one is a locked transaction QUBO instance, denoted as iQ_L , and the second is a remaining transaction QUBO instance, denoted as iQ_{rmt} .

Let's define the locked transaction iter-QUBO instance first:

$$iQ_L = \sum_{\substack{j=\{1,\dots,k\}, \\ i=I^D(I_j^t)}} w_{I_j^t} X_{i,j,0} \quad \text{where, } w_{I_j^t} = \frac{(k+1)^{I_j^t-1}}{(k+1)^R} \quad (10)$$

The iter-QUBO instance for the remaining transactions can be retrieved from the form: $iQ_{rmt} = \min X^T Q X$ by using the hamiltonian H on adding all formula's $\tilde{A}$, $\tilde{B}$, $\tilde{C}$ and $\tilde{D}$.

On adding both the QUBO instances, we get the iter-QUBO as

$$\text{iter-QUBO} = \sum_{\substack{j=\{1,\dots,k\}, \\ i=I^D(I_j^t)}} \underbrace{w_{I_j^t} X_{i,j,0}}_{iQ_L} + \underbrace{\tilde{A} + \tilde{B} + \tilde{C} + \tilde{D}}_{iQ_{rmt}} \quad (11)$$

3.3.6 Valid iterations. To determine the required valid iterations to solve the QUBO as a subproblem, we proceed as follows:

Let C_k be the set of all possible k combinations of the transactions, and $\mathcal{I}$ be the set of valid iterations, defined as

$$C_k = \{\mathcal{T}^t \subseteq T \mid |\mathcal{T}^t| = k\}$$

Then the set of valid locked transactions would be,

$$\mathcal{I} = \left\{ \mathcal{T}^t \in C_k \mid \begin{array}{l} \forall (t_i, t_j) \in \mathcal{B}^t, \{t_i, t_j\} \notin \mathcal{T}^t, \\ \text{and } \forall t_i^r \in \mathcal{T}^t, S_{\{i,j\}}^r \neq \emptyset \end{array} \right\} \quad (12)$$

The required number of iterations to solve the original QUBO iteratively would be equal to

$$\text{Number of iterations}(\#iter) = |\mathcal{I}|$$

For the example E_2 ,

$$C_k = \{(t_1, t_2), (t_1, t_3), (t_1, t_4), (t_1, t_5), (t_2, t_3), (t_2, t_4), (t_2, t_5), (t_3, t_4), (t_3, t_5), (t_4, t_5)\}, \quad k = 2$$

Iterations with locked transactions $\mathcal{L}_4^t = (t_1, t_5)$, $\mathcal{L}_5^t = (t_2, t_3)$ would not be valid, as these are blocking transactions.

In this iteration, $\mathcal{L}_6^t = (t_2, t_4)$, $\mathcal{L}_8^t = (t_3, t_4)$ the transaction t_5 would not have a valid start time; hence, this iteration would be discarded. So, a set of locked transactions as valid iterations are:

$$\mathcal{I} = [(t_1, t_2), (t_1, t_3), (t_1, t_4), (t_2, t_5), (t_3, t_5), (t_4, t_5)]$$

Therefore, the example problem would take $|\mathcal{I}| = 6$ on solving the QUBO problem as a subproblem.

4 Experiments

4.1 Experiment Setup

Experiments were conducted using both gate-based quantum simulators and D-Wave's simulated annealing algorithm. We employed optimization algorithms such as simulated annealing and the QAOA. For simulated annealing, we utilized the dwave-samplers==1.4.0 library. For gate-based quantum simulations, we used Qiskit libraries with the following versions: • qiskit==1.4.2 • qiskit-aer==0.15.1 • qiskit-optimization==0.6.1 • qiskit-algorithms==0.3.1 All experiments were carried out on a local system, specifically an Apple MacBook Pro equipped with an Apple M2 Pro chipset and 16GB RAM, operating on macOS Sequoia == 15.4.1.

4.2 Results and Discussion

To rigorously evaluate the performance of our proposed approach, we conducted experiments using varying sets of transactions within a schedule, alongside diverse transaction lengths. The objective was to create problem instances with sufficient complexity, pushing the boundaries of what current state-of-the-art quantum optimisation technologies can solve efficiently.

In Table 2, we present the experimental results obtained using the Quantum Approximate Optimization Algorithm (QAOA) on a gate-based quantum simulator, implemented via the Qiskit framework using the COBYLA optimizer and setting reps=2. Solving Quadratic Unconstrained Binary Optimization (QUBO) problems with even a modest number of binary variables remains a significant challenge for current quantum simulators due to hardware and algorithmic limitations.

n	k	lengths	blocking trxn	R	#bv(Q)	Q	locked trxn	#bv(i-Q)	i-Q	#iter	Time(Q)	Time(iQ)
3	2	[1,2,1]	(t_1, t_3)	2	10	39	(t_1, t_2)	3	3	2	2.648	0.274
3	2	[1,1,1]	(t_1, t_2)	2	12	46	(t_1, t_3)	4	5	2	9.275	0.65
3	2	[1,2,2]	(t_1, t_3)	3	14	73	(t_1, t_2)	3	3	2	112.27	1.05
3	2	[2,2,2]	(t_1, t_2)	4	18	119	(t_1, t_3)	4	5	2	timeout	0.9265
4	2	[2,3,2,3]	(t_1, t_2)	5	28	252	(t_1, t_3)	6	10	5	timeout	1.8
4	2	[1,2,3,4]	(t_1, t_2)	5	28	236	(t_1, t_3)	7	16	4	timeout	2.775
4	2	[2,1,4,5]	$\{(t_1, t_2), (t_2, t_4)\}$	6	32	326	(t_2, t_3)	8	22	3	timeout	2.15
4	2	[2,1,3,5]	(t_1, t_3)	6	34	335	(t_1, t_2)	8	22	5	timeout	11.68
4	2	[2,3,4,5]	$\{(t_1, t_2), (t_2, t_4)\}$	8	44	628	(t_2, t_3)	10	35	4	timeout	15
4	2	[5,4,3,7]	$\{(t_1, t_3), (t_2, t_4)\}$	11	48	1111	(t_2, t_3)	12	48	4	timeout	70.6
4	2	[9,4,3,6]	$\{(t_1, t_2), (t_2, t_3)\}$	13	68	1476	(t_2, t_4)	15	88	4	timeout	1645.89
5	2	[2,1,1,2,2]	$\{(t_1, t_3), (t_2, t_4)\}$	4	34	281	(t_1, t_2)	13	51	9	timeout	428.54
5	3	[1,2,1,1,2]	$\{(t_1, t_2), (t_3, t_5)\}$	3	39	336	(t_1, t_3, t_4)	9	18	4	timeout	12.32

Table 2: In this table, n is the number of transactions, k is the number of available machines, lengths are the length of the respective transactions, R is the maximum runtime of the machines, #bv(Q) and Q represent the number of binary variables and the size of the full QUBO, locked transactions are those locked at $t = 0$ in the optimal solution, #bv(i-Q) and i-Q represent the number of binary variables and the size of the iterative QUBO, #iter is the number of iterations, Time(Q) is time for full QUBO, and Time(iQ) is time for iterative QUBO. Timeout is set at 30 minutes.

The experiments with QAOA reveal a clear trade-off between problem size and the amount of pruning delivered by our locking transactions at the time $s = 0$ strategy. Starting with three transactions on two machines, progressively richer lengths and conflicts drive the full QUBO size (Q) from 39 to 1476, while the number of binary variables rises only from 10 to 68.

As soon as locked transactions are introduced, the iterative sub-QUBO routine compresses the active variable set in Table 2 column (#bv(i-Q)) by roughly 60–85%, and the time required for iter-QUBO is significantly reduced in comparison to full QUBO. Despite the computational limitation, our results demonstrate the feasibility of solving non-trivial QUBO problems on the quantum simulator, providing valuable insights for advancing quantum-enhanced optimization in transaction scheduling.

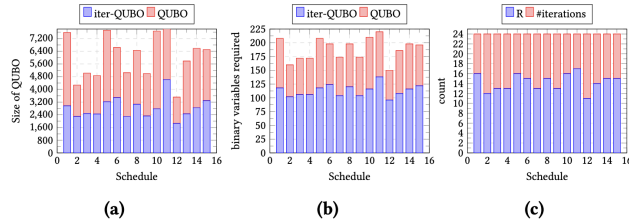


Figure 4: The above figures compare QUBO and iter-QUBO for a schedule having $n = 8$ transactions with $|M| = 2$ available machines, which has been solved using the simulated annealing algorithm. The figure shows the variation in size of QUBO and the variation in required binary variables for QUBO. Iteration for scheduling required 24 iterations only. It has 4 pairs of conflicting transactions.

As illustrated in Figure 4, Figure 5 and Table 3, we focus on involving a set of 8, 10, and 11 transactions using simulated annealing from D-Wave’s Ocean-SDK framework. These transactions were subjected to different runtime distributions, thereby varying the underlying complexity of the resulting QUBO formulations. Our

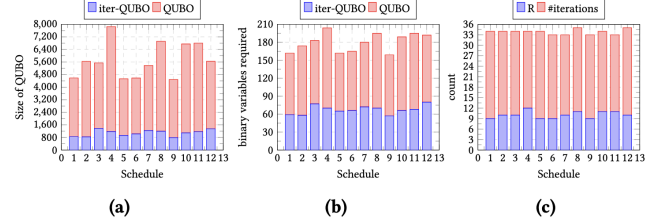


Figure 5: This set of transactions, having $n = 8$ transactions with $|M| = 3$ available machines, was solved using the simulated annealing algorithm. Iteration for scheduling required 34 iterations only. It has 4 pairs of conflicting transactions. Figure 5c shows the maximum runtime for each corresponding schedule.

experiments demonstrate that the strategy of locking k transactions at time $t = 0$, and consequently solving reduced subproblems iteratively, offers a powerful means to manage the overall complexity. This method significantly alleviates the qubit requirements, making the optimization problem more tractable for near-term quantum devices and simulators.

Furthermore, as the number of available machines (or resources) increases, our approach naturally requires a greater number of iterations to accommodate all possible scheduling combinations. However, this increase in iterations is offset by a pronounced reduction in the qubit requirement, due to a higher number of transactions being locked in advance. Thus, our results highlight a compelling trade-off between computational steps and quantum resource consumption, underscoring the practical viability of the proposed method in realistic transaction scheduling scenarios.

To thoroughly evaluate the proposed transaction scheduling method using simulated annealing, we generated a diverse set of transaction schedules with varying transaction lengths. As depicted in Figure 6a and Figure 6b, each schedule consists of transactions

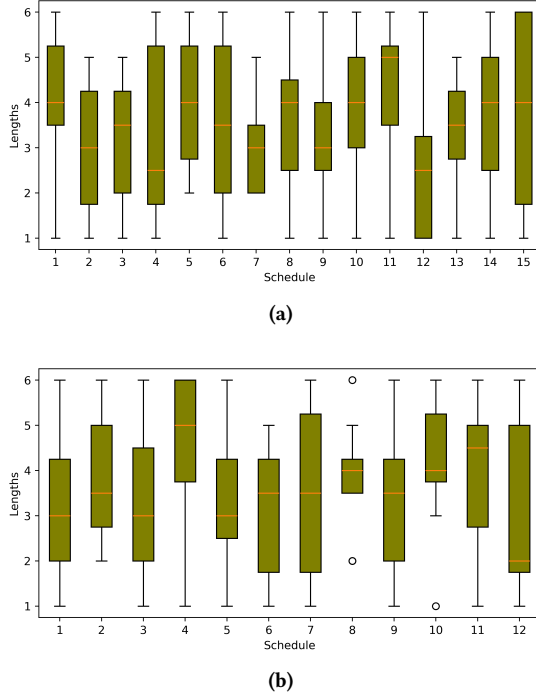


Figure 6: Transaction length variation for experiments in Figure 4 and Figure 5.

with lengths sampled across a range 1 to 6, ensuring sufficient variability and complexity in the problem space. The boxplot highlights the distribution of transaction lengths across multiple schedules, illustrating both moderate and high variances. This heterogeneity in transaction durations is critical for our proposed method, particularly when scaling to more complex and realistic scheduling scenarios.

n	k	$ \mathcal{B}^t $	R	$\#bv(Q)$	Q	$\#bv(i-Q)$	$i-Q$	$\#iter$
10	3	5	11	267	10176	130	3428	82
10	3	5	10	240	8466	112	2373	83
10	3	4	20	450	29829	192	7151	90
11	4	5	10	332	14622	133	3456	185
11	4	5	9	312	11556	132	2656	180
11	4	5	10	344	14736	130	2801	185
11	3	4	19	495	30678	268	12253	131

Table 3: In this table, n is the number of transactions, k is the number of available machines, $|\mathcal{B}^t|$ represents the number of conflicting transactions, R is the maximum runtime of the machines, $\#bv(Q)$ and Q represent the number of binary variables and the size of the full QUBO, $\#bv(i-Q)$ and $i-Q$ represent the binary variable and the size of the iter-QUBO, and $\#iter$ represents the number of iterations required to solve QUBO iteratively.

5 Conclusions

In this study, we presented a hybrid quantum-classical approach for solving the transaction scheduling problem under conflict constraints. By introducing the idea of locking k transactions at $t = 0$ and formulating reduced QUBO subproblems, we demonstrated significant improvements in the solvability of complex scheduling instances. Our locking transaction method can consistently shrink the QUBO search space without sacrificing optimality.

Experiments using simulated annealing and QAOA-based quantum simulations validated the effectiveness of the method, particularly highlighting the advantage of reducing qubit requirements through transaction locking. As the complexity of the schedules increased, our iterative approach maintained a manageable optimization size, enabling the solution of otherwise intractable problems even on near-term quantum simulators.

For every experiment in Table 2, fixing just k transactions cuts the effective binary-variable count ($\#bv(i-Q)$), 60 – 85% and, in turn, allows the reduced iter-QUBOs to be solved on gate-model simulators with QAOA in only a handful of iterations. With our proposed method, iter-QUBO requires significantly less time and resources to solve a full QUBO iteratively. These results suggest that transaction locking is a practical bridge between today’s NISQ machines and real-world scheduling workloads when the full formulation is beyond the quantum hardware budget.

Acknowledgments

This work is funded by the German Federal Ministry of Education and Research within the funding program quantum technologies—from basic research to market—contract number 13N16090.

References

- [1] Emile H. L. Aarts and Jan H. M. Korst. 1988. *Statistical cooling: A general approach to combinatorial optimization problems*. Wiley-Interscience.
- [2] Yuta Atohe, Masashi Tawada, and Nozomu Togawa. 2022. Hybrid Annealing Method Based on subQUBO Model Extraction With Multiple Solution Instances. *IEEE Trans. Comput.* 71, 10 (Oct. 2022), 2606–2619. doi:10.1109/tc.2021.3138629
- [3] Diogo Barbosa, Le Gruenwald, Laurent D’Orazio, and Jorge Bernardino. 2024. QRLIT: Quantum Reinforcement Learning for Database Index Tuning. *Future Internet* 16, 12 (Nov. 2024), 439. doi:10.3390/fi16120439
- [4] Tim Bittner and Sven Groppe. 2020. Avoiding blocking by scheduling transactions using quantum annealing. In *Proceedings of the 24th International Database Engineering & Applications Symposium*. ACM, 1–10. doi:10.1145/3410566.3410593
- [5] Tim Bittner and Sven Groppe. 2020. Hardware Accelerating the Optimization of Transaction Schedules via Quantum Annealing by Avoiding Blocking. *Open Journal of Cloud Computing (OJCC)* 7, 1 (2020), 1–21. http://nbn-resolving.de/urn:nbn:de:101:1-2020112218332015343957
- [6] Kostas Blekos, Dean Brand, Andrea Ceschini, Chiao-Hui Chou, Rui-Hao Li, Komal Pandya, and Alessandro Summer. 2023. A review on Quantum Approximate Optimization Algorithm and its variants. *arXiv preprint arXiv:2306.09198* (2023).
- [7] Thomas M Connolly and Carolyn E Begg. 2005. *Database systems: a practical approach to design, implementation, and management*. Pearson Education.
- [8] Tobias Fankhauser, Marc E. Soler, Rudolf Marcel Fuchsli, and Kurt Stockinger. 2023. Multiple Query Optimization Using a Gate-Based Quantum Computer. *IEEE Access* 11 (2023), 114031–114043. doi:10.1109/access.2023.3324253
- [9] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028* (2014).
- [10] Edward Farhi, Jeffrey Goldstone, Sam Gutmann, and Michael Sipser. 2000. Quantum computation by adiabatic evolution. *arXiv preprint quant-ph/0001106* (2000).
- [11] Maja Franz, Tobias Winker, Sven Groppe, and Wolfgang Mauere. 2024. Hype or Heuristic? Quantum Reinforcement Learning for Join Order Optimisation. In *IEEE International Conference on Quantum Computing and Engineering (QCE)*. https://doi.org/10.1109/QCE60285.2024.00055
- [12] Kristin Fritsch and Stefanie Scherzinger. 2023. Solving Hard Variants of Database Schema Matching on Quantum Computers. *Vldb* 16, 12 (Aug. 2023), 3990–3993. doi:10.14778/3611540.3611603

- [13] Sven Groppe and Jinghua Groppe. 2021. Optimizing Transaction Schedules on Universal Quantum Computers via Code Generation for Grover's Search Algorithm. In *Proceedings of the 25th International Database Engineering & Applications Symposium (IDEAS)*, Montreal, QC, Canada. <https://doi.org/10.1145/3472163.3472164>
- [14] Lov K Grover. 1996. A fast quantum mechanical algorithm for database search. *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC '96)* (1996), 212–219.
- [15] Le Gruenwald, Tobias Winker, Umut Çalkıylmaz, Jinghua Groppe, and Sven Groppe. 2023. Index Tuning with Machine Learning on Quantum Computers for Large-Scale Database Applications. In *QDSM@VLDB, Vancouver, Canada*. <https://ceur-ws.org/Vol-3462/QDSM5.pdf>
- [16] Lester Ingber. 1993. Simulated annealing: Practice versus theory. *Mathematical and Computer Modelling* 18, 11 (1993), 29–57.
- [17] Skander Kazdaghi, Iordanis Kerenidis, Jens Kieckbusch, and Philip Teare. 2024. Improved clinical data imputation via classical and quantum determinantal point processes. *eLife* 12 (May 2024). doi:10.7554/elife.89947
- [18] Manish Kesarwani and Jayant R. Haritsa. 2024. Index Advisors on Quantum Platforms. *VLDB* 17, 11 (July 2024), 3615–3628. doi:10.14778/3681954.3682025
- [19] Scott Kirkpatrick, Charles D Gelatt, and Mario P Vecchi. 1983. Optimization by simulated annealing. *Science* 220, 4598 (1983), 671–680.
- [20] Florian Kittelmann, Pavel Sulimov, and Kurt Stockinger. 2024. QardEst: Using Quantum Machine Learning for Cardinality Estimation of Join Queries. In *Q-Data (SIGMOD/PODS '24)*. ACM, 2–13. doi:10.1145/3665225.3665444
- [21] Didem Kochan, Zheng Zhang, and Xiu Yang. 2022. A Quantum-Inspired Hamiltonian Monte Carlo Method for Missing Data Imputation. In *Proceedings of Mathematical and Scientific Machine Learning (Proceedings of Machine Learning Research, Vol. 190)*, Bin Dong, Qianxiao Li, Lei Wang, and Zhi-Qin John Xu (Eds.). PMLR, 17–32. <https://proceedings.mlr.press/v190/kochan22a.html>
- [22] Tuodu Li, Gongsheng Yuan, Chang Yao, Meng Shi, Ziyue Wang, Ling Qian, and Jiaheng Lu. 2024. Quantum Storage Design for Tables in RDBMS. In *QDSM@VLDB, Guangzhou, China*. <https://vldb.org/workshops/2024/proceedings/QDSM/QDSM.4.pdf>
- [23] Amana Liaqat, Ahmed Darwish, Adrian Roman, and Stephen DiAdamo. 2025. QAOA in Quantum Datacenters: Parallelization, Simulation, and Orchestration. *arXiv preprint arXiv:2503.06233* (2025).
- [24] Tim Littau, Ziyu Li, and Rihan Hai. 2024. Quantum Data Structures for Enhanced Database Performance. In *QDSM@VLDB, Guangzhou, China*. <https://vldb.org/workshops/2024/proceedings/QDSM/QDSM.6.pdf>
- [25] Nitin Nayak, Jan Rehfeld, Tobias Winker, Benjamin Warnke, Umut Çalkıylmaz, and Sven Groppe. 2023. Constructing Optimal Bushy Join Trees by Solving QUBO Problems on Quantum Hardware and Simulators. In *BiDEDE, Seattle, WA, USA*. <https://doi.org/10.1145/3579142.3594298>
- [26] Nitin Nayak, Manuel Schönberger, Valter Uotila, Zhengtong Yan, Sven Groppe, Jiaheng Lu, and Wolfgang Mauerer. 2024. QC'E24 Tutorial: Quantum Annealing – Emerging Exploration for Database Optimization. *arXiv arXiv:2411.04638* (2024). doi:10.48550/ARXIV.2411.04638
- [27] Nitin Nayak, Tobias Winker, Umut Çalkıylmaz, Sven Groppe, and Jinghua Groppe. 2024. Quantum Join Ordering by Splitting the Search Space of QUBO Problems. *Datenbank-Spektrum* 24, 1 (2024), 21–32. <https://doi.org/10.1007/s13222-024-00468-3>
- [28] Tatsuya Noguchi, Keisuke Fukada, Siya Bao, and Nozomu Togawa. 2025. Sub-QUBO Annealing Based on Efficient Binary Variable Selection for Combinatorial Optimization Problems with One-Hot Constraints. In *2025 IEEE International Conference on Consumer Electronics (ICCE)*. IEEE, 1–2.
- [29] M. Rajasekhar and K. Rajan. 2010. A hybrid simulated annealing algorithm for solving permutation flow shop scheduling problems. In *2010 International Conference on Industrial Engineering and Operations Management*. 1–5.
- [30] Claudio Sanavio, Simone Tibaldi, Edoardo Tignone, and Elisa Ercolessi. 2024. Quantum Circuit for Imputation of Missing Data. *IEEE Transactions on Quantum Engineering* 5 (2024), 1–12. doi:10.1109/tqe.2024.3447875
- [31] Pranshi Saxena, Ibrahim Sabek, and Federico Spedalieri. 2024. Constrained Quadratic Model for Optimizing Join Orders. In *Q-Data (SIGMOD/PODS '24)*. ACM, 38–44. doi:10.1145/3665225.3665447
- [32] Manuel Schönberger, Stefanie Scherzinger, and Wolfgang Mauerer. 2023. Ready to Leap (by Co-Design)? Join Order Optimisation on Quantum Hardware. *Proc. ACM Manag. Data* 1, 1, Article 92, 27 pages.
- [33] Manuel Schönberger, Immanuel Trummer, and Wolfgang Mauerer. 2023. Quantum-Inspired Digital Annealing for Join Ordering. In *VLDB*, Vol. 16.
- [34] Manuel Schönberger, Immanuel Trummer, and Wolfgang Mauerer. 2023. Quantum Optimisation of General Join Trees. In *VLDBW'23*.
- [35] Immanuel Trummer and Christoph Koch. 2016. Multiple query optimization on the D-Wave 2X adiabatic quantum computer. *VLDB* 9, 9 (May 2016), 648–659. doi:10.14778/2947618.2947621
- [36] Immanuel Trummer and Davide Venturelli. 2024. Leveraging Quantum Computing for Database Index Selection. In *Q-Data (SIGMOD/PODS '24)*. ACM, 14–26. doi:10.1145/3665225.3665445
- [37] Valter Uotila. 2024. SQL2Circuits: Estimating Metrics for SQL Queries with a Quantum Natural Language Processing Method. *arXiv:2306.08529* [cs.DB] <https://arxiv.org/abs/2306.08529>
- [38] Valter Uotila. 2025. Left-Deep Join Order Selection with Higher-Order Unconstrained Binary Optimization on Quantum Computers. *arXiv:2502.00362* [quant-ph] <https://arxiv.org/abs/2502.00362>
- [39] Peter J. M. Van Laarhoven and Emile H. L. Aarts. 1987. Simulated annealing: Theory and applications. *Mathematics and its Applications* 37 (1987).
- [40] Martin Vogrin, Rok Vogrin, Sven Groppe, and Jinghua Groppe. 2024. Supervised Learning on Relational Databases with Quantum Graph Neural Networks. In *QDSM@VLDB, Guangzhou, China*. <https://vldb.org/workshops/2024/proceedings/QDSM/QDSM.5.pdf>
- [41] Tobias Winker, Sven Groppe, Valter Uotila, Zhengtong Yan, Jiaheng Lu, Maja Franz, and Wolfgang Mauerer. 2023. Quantum Machine Learning: Foundation, New Techniques, and Opportunities for Database Research. In *SIGMOD*. <https://doi.org/10.1145/3555041.3589404>
- [42] Tobias Winker, Umut Çalkıylmaz, Le Gruenwald, and Sven Groppe. 2023. Quantum Machine Learning for Join Order Optimization using Variational Quantum Circuits. In *BiDEDE, Seattle, WA, USA*. <https://doi.org/10.1145/3579142.3594299>
- [43] Wending Zhao and Gaoxiang Tang. 2025. Clustering-Based Sub-QUBO Extraction for Hybrid QUBO Solvers. *arXiv preprint arXiv:2502.16212* (2025).
- [44] Umut Çalkıylmaz, Sven Groppe, Jinghua Groppe, Tobias Winker, Stefan Prestel, Farida Shagieva, Daanish Arya, Florian Preis, and Le Gruenwald. 2023. Opportunities for Quantum Acceleration of Databases: Optimization of Queries and Transaction Schedules. *Proc. VLDB Endow.* 16, 9 (2023), 2344–2353. <https://doi.org/10.14778/3598581.3598603>
- [45] V Černý. 1985. Thermodynamical approach to the traveling salesman problem: An efficient simulation algorithm. *Journal of Optimization Theory and Applications* 45, 1 (1985), 41–51.